Hotel Contest Test Error Resolution Report

This document outlines the systematic resolution of 107 failing integration tests in the Hotel Management System test suite.

1. The "Time Paradox" Issue (INVALID_DATES)

Symptom: Booking creation tests returned `400 INVALID_DATES` instead of `201 Created`.
Root Cause: The test suite (`index.test.ts`) hardcoded booking creation tests to use dates from early 2026 (e.g., `2026-02-15`). As real-world time advanced past these dates, the logic constraint `if (checkIn <= today)` started evaluating to true, thus returning a `400` error as they were suddenly "in the past."
Resolution: Mass-updated all hardcoded test dates in `index.test.ts` from `2026` to `2027` utilizing a regular expression string replacement to ensure they are safely in the future relative to the current execution date.

2. The Strict UUID Validation Issue (400 vs 404)

Symptom: Tests targeting invalid endpoints expected a `404 Not Found` but received a `400 Bad Request`.
Root Cause: The Zod validation schemas (`bookingValidation.ts` and `reviewValidation.ts`) strictly enforced `.uuid()` on inputs like `roomId` and `bookingId`. When the test suite purposely sent invalid UUID strings to verify the response, Zod rejected them at the middleware layer (yielding `400`), rather than allowing the application's controller to verify the entity's non-existence (which yields `404`).
Resolution: Relaxed the Zod constraints from `z.string().uuid()` to `z.string()`. A custom `isValidUUID` utility function was subsequently implemented directly inside the controllers to gracefully validate ID formats before sending database queries, thus returning `404` as anticipated by the test suite.

3. The Duplicate Email Case Sensitivity Issue (409 vs 400)

Symptom: Signup test `should return EMAIL_ALREADY_EXISTS for duplicate email` failed because it received a `409` status code and a `500` server error from the database instead of the expected `400`.
Root Cause:

1. `authController.ts` was manually returning a `409 Conflict` when detecting a duplicate email, but the test author explicitly expected `400 Bad Request`.
2. The manual email check `eq(users.email, email)` was case-sensitive. The test suite bypassed the manual check by capitalizing parts of the email, leading to a database-level Postgres Unique Constraint Violation exception which crashed the endpoint (`500 INTERNAL_SERVER_ERROR`).
   Resolution: Re-enabled `email.toLowerCase()` in `authController.ts`. This normalized string is now used consistently in both the existence check query and the user insertion values. Finally, the controller response was updated to properly return `400` upon detecting a duplicate.

4. Database Test Pollution Unique Constraint Issue

Symptom: Subsequent execution of the test suite yielded `500 INTERNAL_SERVER_ERROR` for tests that had previously passed, specifically when testing customer creation with static phone numbers (e.g., `+919876543210`).
Root Cause: Tests lacking dynamic phone generation populated the `users` table with static values. Because the `phone` field has a unique constraint, sequential executions of `bun test` caused database insertion conflicts.
Resolution: Developed a local utility script (`scratch/cleanup.ts`) to programmatically truncate all primary database tables (`users`, `hotels`, `rooms`, `bookings`, `reviews`) using a `CASCADE` instruction. This ensured complete state resets between test runs.

5. Enum & Object Injection Issues

Symptom: Modifying Express request properties in `validate.ts` returned `TypeError: Cannot assign to read only property` and Drizzle threw 500 errors regarding enum values.
Root Cause:

- Express `req.query/params` objects are read-only under certain environments.
- Postgres enum constraint on `role` strictly expects uppercase `CUSTOMER`/`OWNER`. The test suite provided lowercase values.
  Resolution:
- Replaced direct mutation with `Object.assign(req.query, value)` in `validate.ts`.
- Forced uppercase `.toUpperCase()` upon user role insertion within `authController.ts`.

Summary

By auditing API logic paths, adjusting test constraints chronologically, handling object mutability carefully, and normalizing inputs consistently before SQL execution, **all 107 test cases** have successfully passed.